Задание на проект по проектированию базы данных

Цель проекта: спроектировать и реализовать базу данных для выбранной предметной области — от сбора требований до работающей БД в конкретной СУБД. Акцент на корректность модели данных, нормализацию и соответствие бизнес-правилам.

Выбор темы

Выбери одну из 25 предметных областей. Тема фиксируется у преподавателя и не меняется в ходе проекта.

Этап 1. Сбор и анализ требований

Что нужно сделать:

1. **Изучи предметную область** по предоставленному описанию. Выпиши ключевые процессы и ограничения (например: «нельзя бронировать занятый столик», «один учитель не может вести два урока одновременно»).
2. **Сформулируй роли пользователей** (минимум 2): администратор, сотрудник, клиент. Для каждой роли — 3–5 типичных действий.
3. **Составь список бизнес-правил** (5–7 правил), которые обязательно должны соблюдаться в БД (например: «дата возврата не может быть раньше даты выдачи», «цена товара не может быть отрицательной»).
4. **Определи состав данных**: какие данные нужны для каждого действия (ФИО, даты, суммы, статусы и т. д.). Оцени объёмы: сколько примерно записей ожидается в основных таблицах через 1 год.
5. **Укажи частоту операций**: какие операции будут самыми частыми (чтение списка, поиск, добавление новых записей) и какие — редкими (массовые обновления, отчёты).
6. **Требования к производительности**: какие запросы должны быть быстрыми (например, поиск свободных номеров, проверка статуса посылки).

Результат:

- Техническое задание (кратко, 2–3 страницы): цели, роли, бизнес-правила, состав данных, объёмы, частота операций, требования к производительности.
- Список сущностей и их атрибутов (в виде таблицы: Сущность → Атрибут → Тип данных → Обязательность (да/нет) → Уникальность (да/нет)).

Этап 2. Концептуальное проектирование

Что нужно сделать:

1. На основе списка сущностей и атрибутов построй **ER-модель в нотации Чена**:
   - прямоугольники — сущности;
   - овалы — атрибуты (подчеркни ключевые атрибуты);
   - ромбы — связи;
   - линии с обозначениями — кардинальности (1:1, 1:M, M:N).
2. Для каждой связи укажи её смысл (например, «Клиент делает Бронирование», «Врач ведёт Приём»).
3. Для M:N-связей явно покажи, что это именно M:N (и позже на этапе логического проектирования она превратится в промежуточную таблицу).
4. Модель не должна зависеть от конкретной СУБД: не указывай типы данных, индексы, партиции.

Результат:

- ER-диаграмма в нотации Чена (в Draw.io, Lucidchart).
- Краткое описание каждой связи и её кардинальности, обоснование, почему выбрана именно такая связь.

Этап 3. Логическое проектирование

Что нужно сделать:

1. **Преобразовать ER-модель в реляционную схему** по формальным правилам:
   a) каждая сущность → таблица;
   b) атрибуты → колонки;
   c) 1:1 и 1:M → внешний ключ в одну из таблиц (для 1:1 — с ограничением UNIQUE);
   d) M:N → промежуточная таблица с двумя внешними ключами;
   e) атрибуты связей → колонки промежуточной таблицы;
   f) слабые сущности → таблицы с составным ключом (ключ родителя + собственный дискриминант);
   g) многозначные атрибуты → отдельные таблицы;
   h) составные атрибуты → несколько колонок (если части используются отдельно);
   i) производные атрибуты → не хранятся (вычисляются в запросах).

2. **Показать реляционную схему всех таблиц** (описание таблиц) - перед нормализацией для каждой таблицы определить все атрибуты, абстрактный тип данных, ключи и ограничения.

3. **Провести нормализацию**:
   a) **1NF:** все атрибуты атомарны, нет повторяющихся групп, нет списков в ячейках, нет нумерованных колонок (телефон1, телефон2…);
   b) **2NF:** нет частичных зависимостей — для таблиц с составным PK каждый неключевой атрибут зависит от всего ключа, а не от его части;
   c) **3NF:** нет транзитивных зависимостей — неключевые атрибуты не зависят друг от друга; справочные данные вынесены в отдельные таблицы.

4. **Построить логическую диаграмму БД** — по уже нормализованной схеме, в реляционной нотации (crow's foot или аналог):
   a) каждая таблица — прямоугольник со списком колонок;
   b) первичные ключи (PK) явно помечены;
   c) внешние ключи (FK) явно помечены;
   d) связи между таблицами показаны линиями с обозначением кардинальности (1:M, M:N);
   e) промежуточные таблицы для M:N-связей — на схеме;
   f) абстрактные типы данных (строка, число, дата) — указаны у колонок.

5. **Определить:**
   a) первичные ключи (PK) для каждой таблицы;
   b) внешние ключи (FK) и правила ссылочной целостности (ON UPDATE / ON DELETE: CASCADE, SET NULL, RESTRICT);
   c) ограничения целостности (UNIQUE, NOT NULL, CHECK).

6. **Убедиться, что все бизнес-правила из этапа 1 отражены в ограничениях БД.** Если правило нельзя выразить через CHECK, UNIQUE или FK — зафиксировать его как «требует триггера или логики приложения» с обоснованием.

Результат:

- **Реляционная схема** — текстовое описание таблиц: колонки, абстрактные типы данных, PK, FK, ограничения (NOT NULL, UNIQUE, CHECK), правила ON DELETE / ON UPDATE для каждого FK. Оформить как набор описаний таблиц.
- **Обоснование нормализации** — для каждой таблицы указать, почему она в 3NF (или если есть исключения — почему они допустимы).
- **Логическая диаграмма БД** — графическая схема нормализованных таблиц, колонок, PK, FK и связей (в Draw.io, Lucidchart, dbdiagram.io или аналоге).
- **Отображение бизнес-правил** — таблица: бизнес-правило → ограничение в БД или пометка «триггер / приложение».

Этап 4. Физическое проектирование

Что нужно сделать (с привязкой к PostgreSQL):

1. Выбери конкретные типы данных для каждой колонки (например, VARCHAR(255), DATE, NUMERIC(10,2), BOOLEAN).
2. Определи стратегии индексирования: какие поля будут в индексах и зачем (поиск, уникальность, сортировка).
3. Если уместно, укажи стратегии партиционирования (по дате, по региону и т. п.) и оцени, когда это понадобится (при каком объёме).
4. Продумай хранение: табличные пространства (если СУБД поддерживает), сжатие, расположение файлов (в общих чертах).
5. Учитывай особенности СУБД: например, в PostgreSQL есть типы JSONB, массивы, диапазоны дат — используй их только если это действительно улучшает модель.

Результат:

- Физическая схема БД: описание таблиц с конкретными типами данных, индексами, стратегиями хранения.
- Обоснование выбора типов данных и индексов: для каких запросов они ускоряют работу.

Этап 5. Реализация

Что нужно сделать:

1. Напиши SQL-скрипты для создания объектов БД:
    - CREATE TABLE для всех таблиц с PK, FK и ограничениями.
    - CREATE INDEX для выбранных полей.
    - При необходимости — CREATE VIEW для типовых отчётов.
2. Подготовь скрипт загрузки начальных данных (INSERT) — минимум 10–20 строк в основных таблицах, чтобы можно было проверить запросы.
3. Напиши набор запросов (SELECT) для проверки бизнес-правил и типовых задач из ТЗ:
    - выборки с фильтрацией и сортировкой;
    - JOIN-запросы (2–3 таблицы);
    - агрегации (COUNT, SUM, AVG);
    - подзапросы или CTE для сложной логики (например, выявление конфликтов расписания).
4. Проверь, что ограничения работают: попробуй вставить некорректные данные (например, дату возврата раньше даты выдачи) — должна быть ошибка.

Результат:

- SQL-скрипт создания БД (таблицы, индексы, представления).
- SQL-скрипт начальных данных.

- Набор SQL-запросов с комментариями: какую задачу решает каждый запрос.
- Краткое описание проверок: какие ограничения протестированы и как.

Демонстрационный кейс

0. Учет книг в библиотеке.

Библиотека ведёт учёт книжного фонда, читателей и выдач. Книги хранятся в залах на стеллажах, у каждого экземпляра — статус: в фонде, на полке, выдана, в ремонте. Читатели берут книги на срок, могут продлевать его и возвращать книги. При нарушении срока возврата начисляется штраф. Выдачу и возврат оформляют сотрудники библиотеки (библиотекари), закреплённые за залами. Администраторы управляют структурой библиотеки (залы, стеллажи, жанры), списывают экземпляры и формируют отчёты. Важно контролировать доступность книг, отслеживать задолженности и не допускать выдачи одного и того же экземпляра дважды.

Задачи:

- свободные книги по жанру и наличию в конкретном зале;
- загрузка залов по дням (количество посещений/выдач);
- количество активных выдач и сумма штрафов по каждому библиотекарю.

ТЕМЫ НА ВЫБОР

1. **Система записи в фитнес-клуб**

   В фитнес-клубе продают абонементы разных типов (с лимитом посещений, сроком действия, набором включённых услуг). Клиенты записываются на групповые занятия и персональные тренировки. У каждого тренера есть расписание, залы не могут использоваться одновременно для разных мероприятий. Нужно организовать учёт клиентов, абонементов, тренеров, залов, занятий и записей — чтобы в веб-интерфейсе можно было бронировать и видеть занятость.

   Задачи: посчитать активных клиентов по типам абонементов, найти свободные залы на дату/время, вывести нагрузку тренеров.

2. **Онлайн-запись к врачу в частной клинике**

   Клиника ведёт расписание врачей: у каждого специалиста свои дни и часы приёма, длительность консультации зависит от специализации. Пациенты записываются на приём, у них могут быть разные полисы. Важно отображать свободные слоты, не допускать двойной записи на одно время и хранить историю приёмов.

   Задачи: показать свободные слоты врача на день, посчитать количество записей по специализациям, вывести историю приёмов пациента.

3. **Учёт успеваемости учебной группы**

   Студенты изучают предметы в рамках семестров, оценки выставляются по разным видам контроля (экзамен, зачёт, курсовая работа). Преподаватели ведут разные предметы, а студенты могут пересдавать неудовлетворительные результаты. В веб-системе нужно отображать успеваемость, рейтинги и статистику.

   Задачи: средний балл студента по семестрам, количество «незачётов» по предмету, рейтинг преподавателей по среднему баллу.

4. **Бронирование столиков в ресторане**

   Ресторан состоит из залов, в каждом — столики разной вместимости и типа (VIP, у окна, обычный). Гости бронируют столики на конкретное время, бронь закрепляется за официантом. В интерфейсе важно показывать свободные места, учитывать тип столика и не допускать пересечений по времени.

Задачи: свободные столики нужной вместимости на время, загрузка залов по дням, количество бронирований на официанта.

5. **Каталог животных в питомнике**
   Питомник ведёт учёт питомцев: каждый относится к породе и виду, имеет владельца, историю прививок и осмотров. Владелец может иметь несколько животных. В веб-приложении нужно отображать каталог, график прививок и историю по каждому питомцу.
   Задачи: список питомцев по породам, график прививок, питомцы одного владельца.

6. **Управление автосервисом**
   Клиенты приезжают на автомобилях, оформляют заказ-наряд, куда включают набор услуг и запчастей. У автомобиля накапливается история ремонтов, у заказ-наряда — статусы (в работе, готов, выдан). В интерфейсе важно видеть стоимость ремонта, статусы и статистику по услугам.
   Задачи: стоимость ремонта по заказ-наряду, количество ремонтов по автомобилям, статистика по услугам.

7. **Платформа объявлений**
   Пользователи публикуют объявления в категориях, которые могут быть вложенными (например, «Транспорт → Автомобили → Легковые»). Другие пользователи оставляют отклики и отзывы, у каждого пользователя есть рейтинг. В веб-сервисе нужна фильтрация по категориям, отображение откликов и рейтингов.
   Задачи: объявления по категории и подкатегориям, количество откликов на объявление, рейтинг пользователей.

8. **Учёт в пункте выдачи заказов (ПВЗ)**
   Посылки поступают в ПВЗ, хранятся в ячейках, выдаются получателям. Операторы фиксируют операции, у посылки есть цепочка статусов. В интерфейсе нужно быстро находить посылку по трек-номеру, видеть её статус и ячейку хранения, а также статистику по операциям.
   Задачи: посылки в конкретной ячейке, количество выданных посылок за день, текущий статус посылки по трек-номеру.

9. **Туристическое агентство**
   Путёвки формируются из отдельных компонентов: отель, перелёт, трансфер, страховка. У каждого компонента свои даты и цена, итоговая стоимость складывается из них. В веб-кабинете клиента важно видеть детали путёвки и общую цену, а в админке — статистику по направлениям и отелям.
   Задачи: итоговая стоимость путёвки, количество туристов по отелям, путёвки с вылетами на конкретную дату.

10. **Управление школьным расписанием**
    Школа составляет расписание уроков: каждый урок проходит в кабинете, его ведёт учитель, класс учится по расписанию. Нельзя ставить один кабинет или учителя на два урока одновременно. В веб-интерфейсе важно визуализировать расписание, подсвечивать конфликты и показывать нагрузку учителей.
    Задачи: конфликты расписания (учитель/кабинет в двух местах), свободные кабинеты на слот, нагрузка учителя по неделям.

11. **Складской учёт интернет-магазина**
    Товары хранятся на складах, поступают поставками, уходят заказами. Нужно знать остатки по складам и историю движений, чтобы в веб-интерфейсе отображать доступность товара, формировать отчёты и сигнализировать о низком остатке.
    Задачи: остатки товара по складам, приход/расход за период, товары с низким остатком.

12. **Бронирование номеров в отеле**
    Номера относятся к категориям (стандарт, люкс и т. д.), бронируются гостями на даты, к брони можно добавить дополнительные услуги. В веб-сервисе важно отображать свободные номера, рассчитывать стоимость брони с учётом услуг и показывать загрузку отеля.
    Задачи: свободные номера на даты, стоимость брони с услугами, загрузка отеля по дням.

13. **Платформа онлайн-курсов**
    Курсы состоят из модулей и уроков, студенты проходят их поэтапно и получают сертификаты. Прогресс фиксируется по каждому уроку. В веб-интерфейсе студент видит свой прогресс, а администратор — статистику завершённости курсов.
    Задачи: процент завершения курса студентом, список незавершённых уроков, сертификаты по курсам.

14. **Учёт в ветеринарной клинике**
    Животные принадлежат владельцам, посещают приёмы, получают назначения и сдают анализы. В веб-кабинете владельца нужно видеть историю приёмов и назначений, а в админке клиники — статистику по врачам и типам приёмов.
    Задачи: история приёмов животного, назначения по приёмам, количество приёмов по врачам.

15. **Управление парковкой**
    Машино-места в зонах имеют статусы (свободно, занято, резерв), клиенты могут иметь абонементы, оплата идёт по времени. В веб-интерфейсе оператора важно видеть загрузку зон, активные абонементы и рассчитывать стоимость стоянки.
    Задачи: свободные места в зоне, стоимость стоянки по времени, активные абонементы.

16. **Управление кинотеатром**
    В залах идут сеансы фильмов, продаются билеты на конкретные места. Цена билета зависит от типа места и сеанса. В веб-интерфейсе зрителя нужно отображать сеансы, схему зала и свободные места, а в админке — выручку и заполняемость.
    Задачи: заполняемость сеанса, выручка по сеансам, свободные места на сеанс.

17. **Платформа коворкинга**
    Рабочие места и переговорные бронируются резидентами, есть тарифы (подписка и почасовая аренда). В веб-кабинете резидента важно видеть свои брони и баланс, а в админке — загрузку мест и статистику по тарифам.
    Задачи: занятость рабочих мест по дням, активные брони, стоимость брони по тарифу.

18. **Управление гостиничным комплексом**
    Корпуса содержат этажи и номера, гости бронируют номера и заказывают услуги (ресторан, spa). В веб-интерфейсе администратора нужно отображать загрузку номеров, рассчитывать стоимость услуг и видеть свободные номера.
    Задачи: загрузка номеров по корпусам, стоимость услуг в брони, свободные номера на дату.

19. **ERP для строительной компании**
    Объекты делятся на этапы, на этапах работают бригады, используются материалы от поставщиков. Затраты агрегируются по этапам и объектам. В веб-интерфейсе менеджера важно видеть сметы, остатки материалов и акты выполненных работ.
    Задачи: затраты по этапам, остатки материалов, акты по объектам.

20. **Управление фестивалем или концертом**
    Артисты выступают на сценах в тайм-слотах, для выступлений нужны технические райдеры, зрители покупают билеты на зоны. В веб-интерфейсе организатора нужно отображать расписание без пересечений, требования райдеров и продажи билетов по зонам.
    Задачи: расписание сцен без пересечений, требования райдеров по артистам, билеты по зонам.

21. **Платформа доставки еды**
    Рестораны предлагают меню, клиенты делают заказы, курьеры доставляют по маршрутам. У заказа есть статусы (принят, в пути, доставлен). В веб-интерфейсе клиента важно видеть статус заказа, а в админке — выручку ресторанов и маршруты курьеров.
    Задачи: заказы по статусам, выручка ресторанов, маршруты курьеров.

22. **Учёт в музыкальной школе**
    Ученики занимаются у преподавателей по предметам в классах, расписание не должно конфликтовать. В веб-интерфейсе администратора важно видеть конфликты в расписании, посещаемость и оплату занятий.
    Задачи: конфликты в расписании, посещаемость учеников, оплата по преподавателям.

23. **Управление стоматологической клиникой**
    Пациенты приходят на приёмы, им оказывают процедуры с использованием материалов, выставляют счета. В веб-кабинете врача нужно видеть расписание и историю приёмов, а в админке — стоимость процедур и остатки материалов.
    Задачи: приёмы по врачам, стоимость процедур, остатки материалов.

24. **Система учёта и обслуживания в велопрокате**
    Велосипеды хранятся на станциях в стойках, клиенты берут их по абонементам или разово, фиксируются поездки и повреждения. В веб-интерфейсе оператора важно видеть доступные велосипеды на станции, статистику поездок и инциденты.
    Задачи: доступные велосипеды на станции, статистика поездок, инциденты по велосипедам.

25. **HR-система предприятия**
    Сотрудники относятся к отделам и должностям, есть иерархия подчинения, фиксируются отпуска, больничные и собеседования. В веб-интерфейсе HR-менеджера нужно отображать структуру подчинения, статистику отпусков и воронку подбора по вакансиям.
    Задачи: структура подчинения (кто кому подчиняется), количество отпусков по отделам, воронка подбора по вакансиям.